Project Report

Applied Machine Learning for Cyber Security

21I-1901 | 21I-1920 | 21I-2752

Table of Contents

1. Introduction **3**

2. Understanding of HELLO_FLOOD Attack & RPL Context **3**

3. Dataset Overview **4**

4. Pipeline and Methodology **4**

4.1 Data Cleaning & Preprocessing: 4

4.1.1 Handled missing values 4

4.1.2 Categorical encoding 5

4.1.3 Feature scaling using StandardScaler 6

4.2 Exploratory Data Analysis: 6

4.2.1 Correlation matrix visualization 6

4.2.2 Class distribution inspection 7

4.3 Balancing Techniques: 8

4.3.1 No SMOTE (Baseline) 8

4.3.2 SMOTE After Split 8

4.3.3 SMOTE Before Split 8

4.3.4 Manual Downsampling 9

4.4 Supervised Models Implemented: 9

4.4.1 Logistic Regression 9

4.4.2 Random Forest Classifier 10

4.4.3 Multi-Layer Perceptron (MLP) 11

4.4.4 LSTM (PyTorch-based) 11

4.4.5 GRU (PyTorch-based) 12

4.5 Unsupervised Technique: 13

4.5.1 Isolation Forest (applied on benign-only trained model) 13

4.6 Evaluation Metrics: 15

4.6.1 Accuracy, Precision, Recall, F1 Score 15

4.6.2 Confusion matrices 15

4.6.3 Visuals: Bar plots, slope graphs 16

5. Summary and Finding **17**

6. Conclusion **18**

Hello_Flood Attack Detection

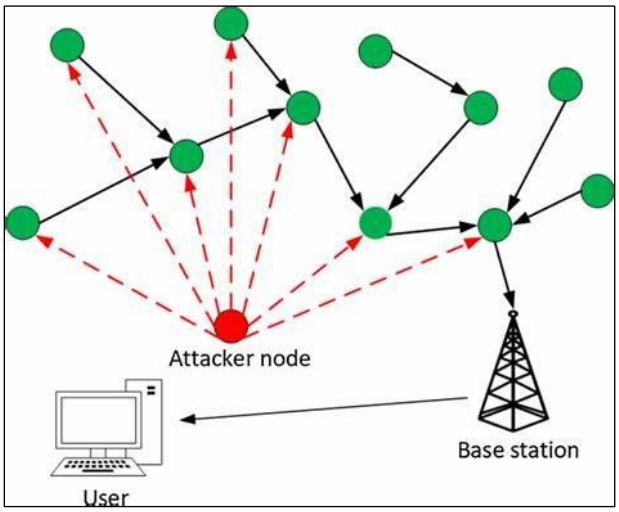
1. Introduction

In the context of increasing adoption of IoT devices, security threats to IoT networks have become a critical concern. One such vulnerability is present in **RPL-based IoT networks**, which are widely used for routing in low-power and lossy networks (LLNs). Among the numerous routing threats targeting RPL, the **HELLO_FLOOD** attack is particularly dangerous due to its ability to exhaust node resources and mislead network topology formation. Our project aims to detect **HELLO_FLOOD attacks** using various **Machine Learning (ML)** and **Deep Learning (DL)** techniques, leveraging a provided labeled dataset to build and evaluate classification models.

2. Understanding of HELLO_FLOOD Attack & RPL Context

The RPL (Routing Protocol for Low-Power and Lossy Networks) forms a Destination Oriented Directed Acyclic Graph (DODAG) to manage routing paths between nodes. **Each node broadcasts DIO (DODAG Information Object) messages** to announce its presence and network attributes.

In a HELLO_FLOOD attack, a malicious node sends **excessive fake HELLO/DIO messages** to all neighboring nodes. These victim nodes believe the attacker is a legitimate and optimal parent, leading them to attempt resource-draining communication. This results in unnecessary battery drain, bandwidth usage, and topology disruption.



3. Dataset Overview

The dataset provided simulates network traffic under normal and HELLO_FLOOD attack scenarios. Each row represents a network packet with features including control packet type (e.g., DIO), packet size, transmission rate, hop count, and other RPL-related metrics.

- **Features**: Numerical and categorical features representing packet metadata and transmission behavior.
- **Target**: Binary label PCKT_LABEL — 0 (Benign), 1 (Attack)
- **Class Distribution**: Initially imbalanced with more benign samples than attack samples.

4. Pipeline and Methodology

4.1 Data Cleaning & Preprocessing:

In any machine learning project, ensuring the quality and consistency of the dataset is crucial. For our HELLO_FLOOD detection task, the data cleaning and preprocessing stage involved the following key steps:

4.1.1 Handled missing values

The dataset was first examined for any missing or null values. Missing values can significantly impact the training of machine learning models, leading to incorrect patterns and reduced accuracy.

```
Null Value Count in Dataset (Top 10):

CWND                          897680
SEQ_NO                        897680
ACK_NO                        897680
isSyn                         897680
isAck                         897680
isFin                         897680
LOCAL_ADDRESS                 897680
SEGMENT_LEN                   897680
REMOTE_ADDRESS                897680
Unnamed: 38                   897680
APP_LAYER_PAYLOAD(Bytes)      863743
TRX_LAYER_PAYLOAD(Bytes)      863743
TRX_LAYER_ARRIVAL_TIME(US)    863743
SEGMENT_ID                    863743
APP_LAYER_ARRIVAL_TIME(US)    863743
RPL_RANK                       33937
RPL_VERSION                    33937
```

Columns containing missing entries were either dropped (if insignificant in number) or imputed using domain-relevant strategies.

```
Columns with 100% missing values: ['LOCAL_ADDRESS', 'REMOTE_ADDRESS', 'CWND', 'SEQ_NO', 'ACK_NO', 'isSyn', 'isAck',
'isFin', 'SEGMENT_LEN', 'Unnamed: 38']
```

```
Columns dropped due to >60% missing values: ['SEGMENT_ID', 'APP_LAYER_ARRIVAL_TIME(US)',
'TRX_LAYER_ARRIVAL_TIME(US)', 'APP_LAYER_PAYLOAD(Bytes)', 'TRX_LAYER_PAYLOAD(Bytes)']
```

In our case, most rows with missing values were removed and imputed.

```
NaN values filled using column mean.

New shape after imputation: (897680, 16)
```

4.1.2 Categorical encoding

Certain columns in the dataset, such as **CONTROL_PACKET_TYPE/APP_NAME**, contained categorical data (e.g., DIO, DAO). These categorical values were not directly usable by machine learning algorithms, which require numerical input.

	PACKET_TYPE	CONTROL_PACKET_TYPE/APP_NAME	SOURCE_ID	DESTINATION_ID	TRANSMITTER_ID	RECEIVER_ID
0	Control_Packet	OSPF_HELLO	SINKNODE-17	Broadcast-0	SINKNODE-17	ROUTER-18
1	Control_Packet	OSPF_HELLO	ROUTER-18	Broadcast-0	ROUTER-18	SINKNODE-17
2	Control_Packet	DIO	SINKNODE-17	Broadcast-0	SINKNODE-17	SENSOR-5
3	Control_Packet	DIO	SINKNODE-17	Broadcast-0	SINKNODE-17	SENSOR-8
4	Control_Packet	DIO	SINKNODE-17	Broadcast-0	SINKNODE-17	SENSOR-10

To convert them, we applied **Label Encoding**, which assigns a unique integer to each categorical label. This transformation preserved the label uniqueness without adding unnecessary dimensionality (as would occur with One-Hot Encoding).

```
Categorical Columns Encoded Successfully.
```

	PACKET_TYPE	CONTROL_PACKET_TYPE/APP_NAME	SOURCE_ID	DESTINATION_ID	TRANSMITTER_ID	RECEIVER_ID
0	0	19	17	0	17	1
1	0	19	0	0	0	18
2	0	17	17	0	17	13
3	0	17	17	0	17	16
4	0	17	17	0	17	3

4.1.3 Feature scaling using StandardScaler

After encoding, the dataset contained a mix of numerical features with varying ranges. Features like packet size or transmission delay could differ by orders of magnitude, leading to biased training in distance-based or gradient-based models. Therefore, we applied **StandardScaler**, a normalization technique that standardizes each feature to have a mean of 0 and a standard deviation of 1. This ensured that all features contributed equally during training, especially benefiting algorithms like Logistic Regression, Neural Networks.

```
Feature scaling completed using StandardScaler.

Scaled Data Shape: (897680, 16)
```

```
Scaled Feature Values:
```

	PACKET_TYPE	CONTROL_PACKET_TYPE/APP_NAME	SOURCE_ID	DESTINATION_ID	TRANSMITTER_ID	RECEIVER_ID	PHY_LAYER_ARRIVAL_TIME(US)	PHY_LAYER_START_TIME(US)	PHY_LAYER_END_TIM
0	-0.197267	1.201228	2.646917	-0.287577	2.470241	-1.702866	-3.364658	-3.364657	-3.3
1	-0.197267	1.201228	-5.558463	-0.287577	-5.192338	1.245818	-3.364658	-3.364657	-3.3
2	-0.197267	0.214690	2.646917	-0.287577	2.470241	0.378558	-3.364635	-3.364635	-3.3
3	-0.197267	0.214690	2.646917	-0.287577	2.470241	0.898914	-3.364635	-3.364635	-3.3
4	-0.197267	0.214690	2.646917	-0.287577	2.470241	-1.355962	-3.364635	-3.364635	-3.3

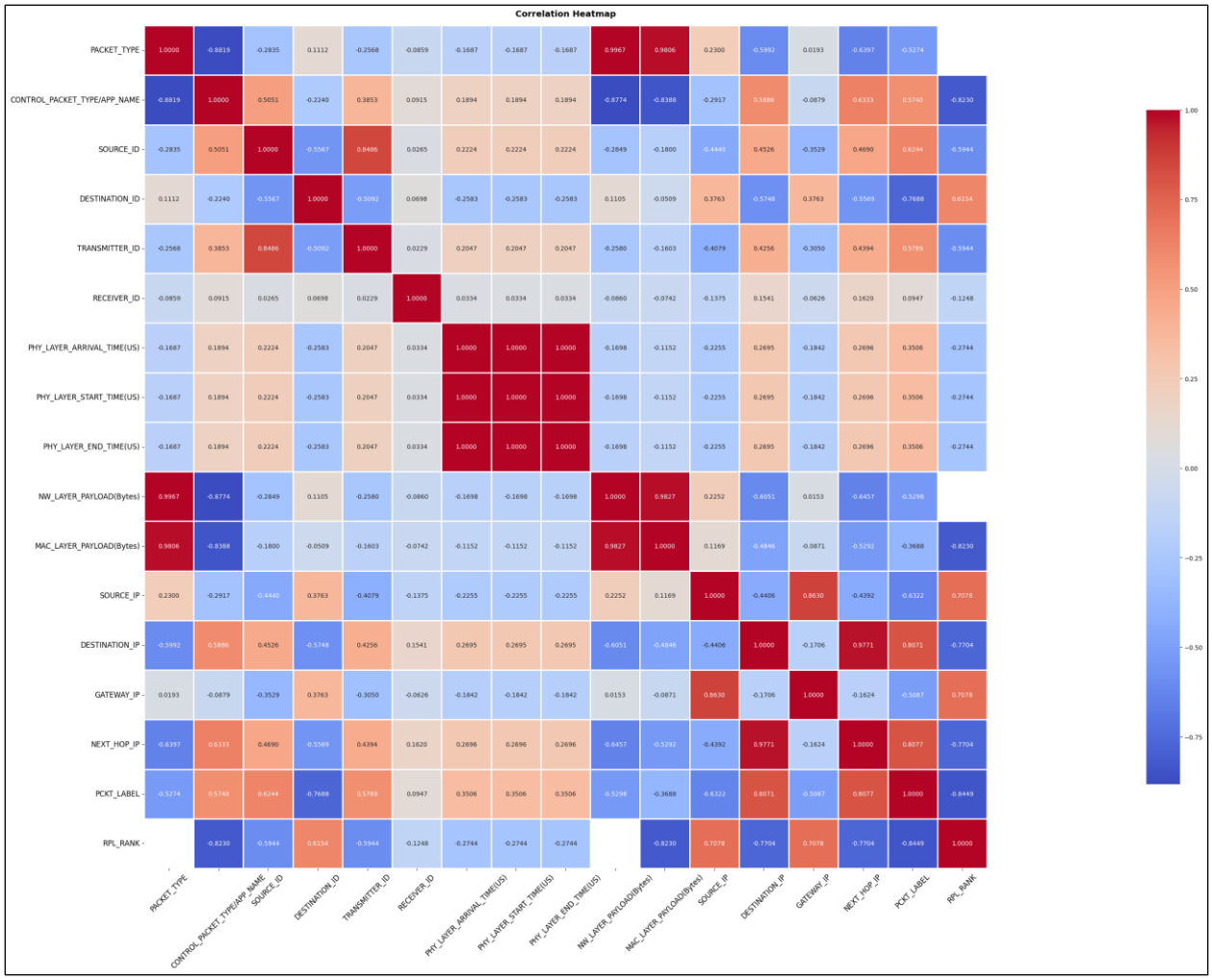
4.2 Exploratory Data Analysis:

Exploratory Data Analysis played a vital role in understanding the underlying structure, relationships, and class imbalance within the HELLO_FLOOD dataset. This step helped inform decisions about preprocessing, model selection, and the need for data balancing techniques.

4.2.1 Correlation matrix visualization

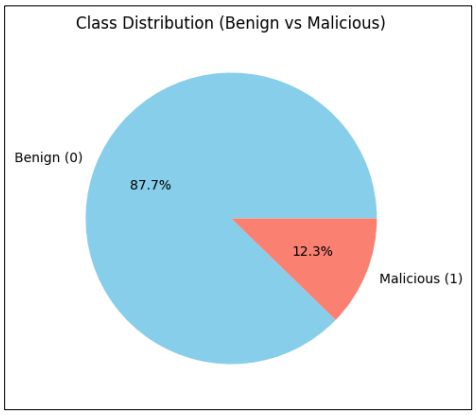
A **correlation matrix heatmap** was plotted using Seaborn to visualize the relationships between numerical features. This matrix displays the Pearson correlation coefficients between each pair of features, ranging from -1 (perfect negative correlation) to +1 (perfect positive correlation). Features with strong positive or negative correlations were identified, helping us understand which variables are likely to influence model predictions. Key insights:

- Features like dpkts and spkts showed high correlation with others, indicating they may be impactful for classification.
- Highly correlated features were retained since tree-based models like Random Forest can handle multicollinearity efficiently.



4.2.2 Class distribution inspection

Before applying any ML/DL model, we inspected the class distribution between **benign (label = 0)** & **malicious (label = 1)** packets. As expected in a real-world IoT attack scenario, the dataset was **highly imbalanced**, with benign samples significantly outnumbering malicious ones.



Understanding this imbalance was essential for:

- Choosing appropriate evaluation metrics (F1 Score, Recall over Accuracy).
- Applying balancing techniques like SMOTE or manual downsampling to improve model sensitivity to minority class (attack).

4.3 Balancing Techniques:

Given the highly imbalanced nature of the HELLO_FLOOD dataset, where benign instances significantly outnumber malicious ones, we explored multiple data balancing strategies to ensure fair learning and prevent model bias. Each technique was implemented in a separate notebook to assess its individual impact on model performance.

4.3.1 No SMOTE (Baseline)

This served as the baseline where no balancing technique was applied. The dataset was split into training and testing sets using stratified sampling to preserve class ratios. Models trained on this setup exhibited **high accuracy but poor recall for the minority class**, clearly demonstrating the impact of imbalance on detection of attacks. This baseline helped measure the gain from later balancing efforts.

4.3.2 SMOTE After Split

In this approach, the original dataset was first split into training (70%) and testing (30%) sets. **SMOTE (Synthetic Minority Oversampling Technique)** was applied **only on the training set**, ensuring the test set remained untouched and realistic.

- This method helps simulate a real-world deployment where models are trained on balanced data but evaluated on true imbalanced distributions.
- It improved **recall** and **F1-score** significantly while avoiding data leakage.

4.3.3 SMOTE Before Split

Here, SMOTE was applied **before** the train-test split. The full dataset was balanced first, and then split into training and testing subsets.

- Both sets had a **uniformly balanced distribution**.
- While this allowed the model to generalize well across both classes, it introduced a potential **data leakage risk** since synthetic samples might appear in both sets.
- Useful for understanding **best-case model behavior**, but not ideal for deployment testing.

4.3.4 Manual Downsampling

Instead of oversampling the minority class, we reduced the size of the **benign (majority) class** by randomly selecting a number of benign samples equal to the number of malicious ones.

- The combined balanced dataset was then shuffled and split.
- This method ensured that the model learned only from **real-world examples** (no synthetic data).
- It yielded consistent and robust performance, especially for simpler models like Logistic Regression and MLP.

```
# Separate benign and malicious
malicious_df = df[df["PCKT_LABEL"] == 1]
benign_df = df[df["PCKT_LABEL"] == 0]

# Find the smaller class count
min_count = min(len(benign_df), len(malicious_df))

# Downsample both to the same size
benign_downsampled = benign_df.sample(n=min_count, random_state=42)
malicious_downsampled = malicious_df.sample(n=min_count, random_state=42)

# Combine and shuffle
balanced_df = pd.concat([benign_downsampled, malicious_downsampled], axis=0).sample(frac=1, random_state=42)

# Check balance
console.print("[bold green]Balanced class distribution (Manual Downsampling):[/bold green]")
console.print(balanced_df["PCKT_LABEL"].value_counts())
```
✓ 0.3s

4.4 Supervised Models Implemented:

To accurately classify packets as either **benign or malicious (HELLO_FLOOD attacks)**, we implemented a variety of supervised learning models ranging from classical machine learning to deep learning architectures. Each model was evaluated using standard metrics (Accuracy, Precision, Recall, F1 Score) and confusion matrices to ensure thorough performance comparison.

4.4.1 Logistic Regression

- A simple, interpretable linear model that predicts probabilities using a sigmoid activation.
- Chosen as a **baseline classifier** due to its ease of implementation and relatively good performance on linearly separable data.
- Worked well after data balancing, especially with **SMOTE After Split** and **Manual Downsampling** techniques.
- Served as a benchmark to compare performance of more complex models.

Technique	Model	Accuracy	Precision	Recall	F1 Score
Baseline	Logistic Regression	0.9998	0.9998	1.0000	0.9999
Baseline	Random Forest	1.0000	1.0000	1.0000	1.0000
Baseline	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
Baseline	LSTM	0.9641	0.9786	0.9805	0.9795
Baseline	GRU	0.9631	0.9701	0.9884	0.9792
Baseline	Isolation Forest	0.9288	0.9250	1.0000	0.9610
SMOTE After	Logistic Regression	0.9999	0.9999	1.0000	0.9999
SMOTE After	Random Forest	1.0000	1.0000	1.0000	1.0000
SMOTE After	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
SMOTE After	LSTM	0.5738	0.5799	0.9350	0.7158
SMOTE After	GRU	0.5730	0.6291	0.6244	0.6267
SMOTE After	Isolation Forest	0.9288	0.9250	1.0000	0.9610
SMOTE Before	Logistic Regression	0.9995	0.9989	1.0000	0.9995
SMOTE Before	Random Forest	1.0000	1.0000	1.0000	1.0000
SMOTE Before	MLP Neural Net	0.9999	0.9999	1.0000	0.9999
SMOTE Before	LSTM	0.9634	0.9770	0.9814	0.9792
SMOTE Before	GRU	0.9640	0.9796	0.9794	0.9795
SMOTE Before	Isolation Forest	0.9288	0.9250	1.0000	0.9610
Downsampling	Logistic Regression	0.9995	0.9990	1.0000	0.9995
Downsampling	Random Forest	1.0000	1.0000	1.0000	1.0000
Downsampling	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
Downsampling	LSTM	0.4988	0.4968	0.6815	0.5747
Downsampling	GRU	0.4984	0.4987	0.8027	0.6152
Downsampling	Isolation Forest	0.7367	0.6551	1.0000	0.7916

4.4.2 Random Forest Classifier

- An ensemble model that builds multiple decision trees and aggregates their predictions (majority voting).
- Highly effective in handling noisy and imbalanced data.
- Produced **perfect or near-perfect performance metrics**, especially under **SMOTE Before Split**, due to its robust nature and ability to model complex patterns.
- Overfitting was monitored and mitigated by evaluating on unaltered test sets.

Technique	Model	Accuracy	Precision	Recall	F1 Score
Baseline	Logistic Regression	0.9998	0.9998	1.0000	0.9999
Baseline	Random Forest	1.0000	1.0000	1.0000	1.0000
Baseline	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
Baseline	LSTM	0.9641	0.9786	0.9805	0.9795
Baseline	GRU	0.9631	0.9701	0.9884	0.9792
Baseline	Isolation Forest	0.9288	0.9250	1.0000	0.9610
SMOTE After	Logistic Regression	0.9999	0.9999	1.0000	0.9999
SMOTE After	Random Forest	1.0000	1.0000	1.0000	1.0000
SMOTE After	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
SMOTE After	LSTM	0.5738	0.5799	0.9350	0.7158
SMOTE After	GRU	0.5730	0.6291	0.6244	0.6267
SMOTE After	Isolation Forest	0.9288	0.9250	1.0000	0.9610
SMOTE Before	Logistic Regression	0.9995	0.9989	1.0000	0.9995
SMOTE Before	Random Forest	1.0000	1.0000	1.0000	1.0000
SMOTE Before	MLP Neural Net	0.9999	0.9999	1.0000	0.9999
SMOTE Before	LSTM	0.9634	0.9770	0.9814	0.9792
SMOTE Before	GRU	0.9640	0.9796	0.9794	0.9795
SMOTE Before	Isolation Forest	0.9288	0.9250	1.0000	0.9610
Downsampling	Logistic Regression	0.9995	0.9990	1.0000	0.9995
Downsampling	Random Forest	1.0000	1.0000	1.0000	1.0000
Downsampling	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
Downsampling	LSTM	0.4988	0.4968	0.6815	0.5747
Downsampling	GRU	0.4984	0.4987	0.8027	0.6152
Downsampling	Isolation Forest	0.7367	0.6551	1.0000	0.7916

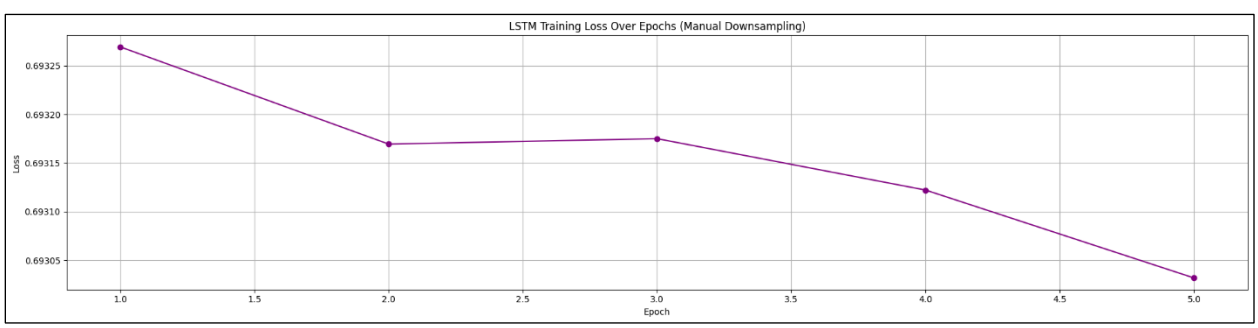
4.4.3 Multi-Layer Perceptron (MLP)

- A fully connected feedforward neural network with one hidden layer (64 neurons) followed by a second hidden layer (32 neurons).
- **Activation:** ReLU; Optimizer: Adam; Loss: Binary Cross Entropy.
- Capable of learning non-linear decision boundaries.
- Showed solid performance improvements post-balancing and captured patterns that classical models missed.

Technique	Model	Accuracy	Precision	Recall	F1 Score
Baseline	Logistic Regression	0.9998	0.9998	1.0000	0.9999
Baseline	Random Forest	1.0000	1.0000	1.0000	1.0000
Baseline	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
Baseline	LSTM	0.9641	0.9786	0.9805	0.9795
Baseline	GRU	0.9631	0.9701	0.9884	0.9792
Baseline	Isolation Forest	0.9288	0.9250	1.0000	0.9610
SMOTE After	Logistic Regression	0.9999	0.9999	1.0000	0.9999
SMOTE After	Random Forest	1.0000	1.0000	1.0000	1.0000
SMOTE After	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
SMOTE After	LSTM	0.5738	0.5799	0.9350	0.7158
SMOTE After	GRU	0.5730	0.6291	0.6244	0.6267
SMOTE After	Isolation Forest	0.9288	0.9250	1.0000	0.9610
SMOTE Before	Logistic Regression	0.9995	0.9989	1.0000	0.9995
SMOTE Before	Random Forest	1.0000	1.0000	1.0000	1.0000
SMOTE Before	MLP Neural Net	0.9999	0.9999	1.0000	0.9999
SMOTE Before	LSTM	0.9634	0.9770	0.9814	0.9792
SMOTE Before	GRU	0.9640	0.9796	0.9794	0.9795
SMOTE Before	Isolation Forest	0.9288	0.9250	1.0000	0.9610
Downsampling	Logistic Regression	0.9995	0.9990	1.0000	0.9995
Downsampling	Random Forest	1.0000	1.0000	1.0000	1.0000
Downsampling	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
Downsampling	LSTM	0.4988	0.4968	0.6815	0.5747
Downsampling	GRU	0.4984	0.4987	0.8027	0.6152
Downsampling	Isolation Forest	0.7367	0.6551	1.0000	0.7916

4.4.4 LSTM (PyTorch-based)

- Implemented using PyTorch with two LSTM layers.
- Designed to learn **sequential dependencies** across a window of 10 packets, simulating time-aware intrusion detection.
- Training was done using a 70/30 split with batch-wise iteration through a DataLoader.



- Performed well under **baseline** and **SMOTE before Split**, but was **less stable under SMOTE after Split** due to potential overfitting to synthetic patterns.

Technique	Model	Accuracy	Precision	Recall	F1 Score
Baseline	Logistic Regression	0.9998	0.9998	1.0000	0.9999
Baseline	Random Forest	1.0000	1.0000	1.0000	1.0000
Baseline	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
Baseline	LSTM	0.9641	0.9786	0.9805	0.9795
Baseline	GRU	0.9631	0.9701	0.9884	0.9792
Baseline	Isolation Forest	0.9288	0.9250	1.0000	0.9610
SMOTE After	Logistic Regression	0.9999	0.9999	1.0000	0.9999
SMOTE After	Random Forest	1.0000	1.0000	1.0000	1.0000
SMOTE After	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
SMOTE After	LSTM	0.5738	0.5799	0.9350	0.7158
SMOTE After	GRU	0.5730	0.6291	0.6244	0.6267
SMOTE After	Isolation Forest	0.9288	0.9250	1.0000	0.9610
SMOTE Before	Logistic Regression	0.9995	0.9989	1.0000	0.9995
SMOTE Before	Random Forest	1.0000	1.0000	1.0000	1.0000
SMOTE Before	MLP Neural Net	0.9999	0.9999	1.0000	0.9999
SMOTE Before	LSTM	0.9634	0.9770	0.9814	0.9792
SMOTE Before	GRU	0.9640	0.9796	0.9794	0.9795
SMOTE Before	Isolation Forest	0.9288	0.9250	1.0000	0.9610
Downsampling	Logistic Regression	0.9995	0.9990	1.0000	0.9995
Downsampling	Random Forest	1.0000	1.0000	1.0000	1.0000
Downsampling	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
Downsampling	LSTM	0.4988	0.4968	0.6815	0.5747
Downsampling	GRU	0.4984	0.4987	0.8027	0.6152
Downsampling	Isolation Forest	0.7367	0.6551	1.0000	0.7916

4.4.5 GRU (PyTorch-based)

- A simplified alternative to LSTM, GRUs retain sequence learning capabilities with fewer parameters.
- Implemented with similar structure and hyperparameters as LSTM.
- Performed well under **baseline** and **SMOTE before Split**, but was **less stable under SMOTE after Split** due to potential overfitting to synthetic patterns
- Ideal for resource-constrained environments due to lower computational cost.

Technique	Model	Accuracy	Precision	Recall	F1 Score
Baseline	Logistic Regression	0.9998	0.9998	1.0000	0.9999
Baseline	Random Forest	1.0000	1.0000	1.0000	1.0000
Baseline	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
Baseline	LSTM	0.9641	0.9786	0.9805	0.9795
Baseline	GRU	0.9631	0.9701	0.9884	0.9792
Baseline	Isolation Forest	0.9288	0.9250	1.0000	0.9610
SMOTE After	Logistic Regression	0.9999	0.9999	1.0000	0.9999
SMOTE After	Random Forest	1.0000	1.0000	1.0000	1.0000
SMOTE After	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
SMOTE After	LSTM	0.5738	0.5799	0.9350	0.7158
SMOTE After	GRU	0.5730	0.6291	0.6244	0.6267
SMOTE After	Isolation Forest	0.9288	0.9250	1.0000	0.9610
SMOTE Before	Logistic Regression	0.9995	0.9989	1.0000	0.9995
SMOTE Before	Random Forest	1.0000	1.0000	1.0000	1.0000
SMOTE Before	MLP Neural Net	0.9999	0.9999	1.0000	0.9999
SMOTE Before	LSTM	0.9634	0.9770	0.9814	0.9792
SMOTE Before	GRU	0.9640	0.9796	0.9794	0.9795
SMOTE Before	Isolation Forest	0.9288	0.9250	1.0000	0.9610
Downsampling	Logistic Regression	0.9995	0.9990	1.0000	0.9995
Downsampling	Random Forest	1.0000	1.0000	1.0000	1.0000
Downsampling	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
Downsampling	LSTM	0.4988	0.4968	0.6815	0.5747
Downsampling	GRU	0.4984	0.4987	0.8027	0.6152
Downsampling	Isolation Forest	0.7367	0.6551	1.0000	0.7916

4.5 Unsupervised Technique:

To complement our supervised learning pipeline, we explored anomaly detection using an **unsupervised learning technique**. This helps validate how well models can identify attacks without labeled training data—simulating real-world scenarios where labels may be unavailable.

4.5.1 Isolation Forest (applied on benign-only trained model)

Isolation Forest identifies anomalies by isolating data points using random partitioning. It assumes that anomalies are **easier to isolate** than normal points and thus require fewer splits.

Implementation Steps:

Training Data:

Trained only on **benign samples** (PCKT_LABEL = 0) to simulate learning the **"normal"** behavior.

```
# ==========================================================================================
# Step 1: Filter benign samples for training
# ==========================================================================================

X_benign = X_scaled_df[y == 0]
X_full = X_scaled_df
y_true = y.reset_index(drop=True)
```

Prediction:

Evaluated across the **entire dataset**, marking samples as anomalous (potential attacks) if they deviated significantly. Predictions:

- -1 = anomaly → mapped to 1 = attack
- 1 = normal → mapped to 0 = benign

```
# ==========================================================================================
# Step 3: Predict on full dataset
# ==========================================================================================

predictions = iso_model.predict(X_full)
# IsolationForest outputs: 1 = normal, -1 = anomaly → convert to: 0 = benign, 1 = attack
y_pred_iso = pd.Series((predictions == -1).astype(int))
```

Evaluation:

Across **Baseline**, **SMOTE After Split**, and **SMOTE Before Split**:

- Isolation Forest performed **surprisingly well**, achieving:
  - **Accuracy ~92.88%**
  - **Precision ~92.5%**
  - **Recall = 100%**
  - **F1 Score ~96.1%**
- This indicates strong anomaly separation under original and synthetically balanced datasets.

However, under the **Manual Downsampling** technique:

- Performance **dropped significantly**:
  - Precision: **~65.5%**
  - F1 Score: **~79.1%**

Likely due to:

- Smaller data size
- Reduced variance in benign training data
- Reduced ability to isolate attack-specific outliers.

Technique	Model	Accuracy	Precision	Recall	F1 Score
Baseline	Logistic Regression	0.9998	0.9998	1.0000	0.9999
Baseline	Random Forest	1.0000	1.0000	1.0000	1.0000
Baseline	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
Baseline	LSTM	0.9641	0.9786	0.9805	0.9795
Baseline	GRU	0.9631	0.9701	0.9884	0.9792
Baseline	Isolation Forest	0.9288	0.9250	1.0000	0.9610
SMOTE After	Logistic Regression	0.9999	0.9999	1.0000	0.9999
SMOTE After	Random Forest	1.0000	1.0000	1.0000	1.0000
SMOTE After	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
SMOTE After	LSTM	0.5738	0.5799	0.9350	0.7158
SMOTE After	GRU	0.5730	0.6291	0.6244	0.6267
SMOTE After	Isolation Forest	0.9288	0.9250	1.0000	0.9610
SMOTE Before	Logistic Regression	0.9995	0.9989	1.0000	0.9995
SMOTE Before	Random Forest	1.0000	1.0000	1.0000	1.0000
SMOTE Before	MLP Neural Net	0.9999	0.9999	1.0000	0.9999
SMOTE Before	LSTM	0.9634	0.9770	0.9814	0.9792
SMOTE Before	GRU	0.9640	0.9796	0.9794	0.9795
SMOTE Before	Isolation Forest	0.9288	0.9250	1.0000	0.9610
Downsampling	Logistic Regression	0.9995	0.9990	1.0000	0.9995
Downsampling	Random Forest	1.0000	1.0000	1.0000	1.0000
Downsampling	MLP Neural Net	1.0000	1.0000	1.0000	1.0000
Downsampling	LSTM	0.4988	0.4968	0.6815	0.5747
Downsampling	GRU	0.4984	0.4987	0.8027	0.6152
Downsampling	Isolation Forest	0.7367	0.6551	1.0000	0.7916

4.6 Evaluation Metrics:

To assess model performance thoroughly and consistently across all techniques and algorithms, the following evaluation methods were employed:

4.6.1 Accuracy, Precision, Recall, F1 Score

For both **supervised** and **unsupervised** models, the following metrics were computed:

- **Accuracy**: Proportion of correctly classified samples out of all samples.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision**: Ability of the model to return only relevant results (true positive rate for predicted positives).

$$Precision = \frac{TP}{TP + FP}$$

- **Recall (Sensitivity)**: Ability to identify all actual positive instances.

$$Recall = \frac{TP}{TP + FN}$$

- **F1 Score**: Harmonic mean of Precision and Recall. It balances both metrics to give a single score.

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

These metrics were critical for judging the model's capability to detect attacks without introducing too many false positives.

4.6.2 Confusion matrices

Each model's predictions were evaluated through **confusion matrices**, which visually represented:

- **True Positives (TP)** – Correctly predicted attacks.
- **True Negatives (TN)** – Correctly predicted benign packets.
- **False Positives (FP)** – Benign packets misclassified as attacks.
- **False Negatives (FN)** – Missed attack packets.

The confusion matrix provided insight into **error distribution** and was especially helpful in understanding model behavior under class imbalance scenarios.

4.6.3 Visuals: Bar plots, slope graphs

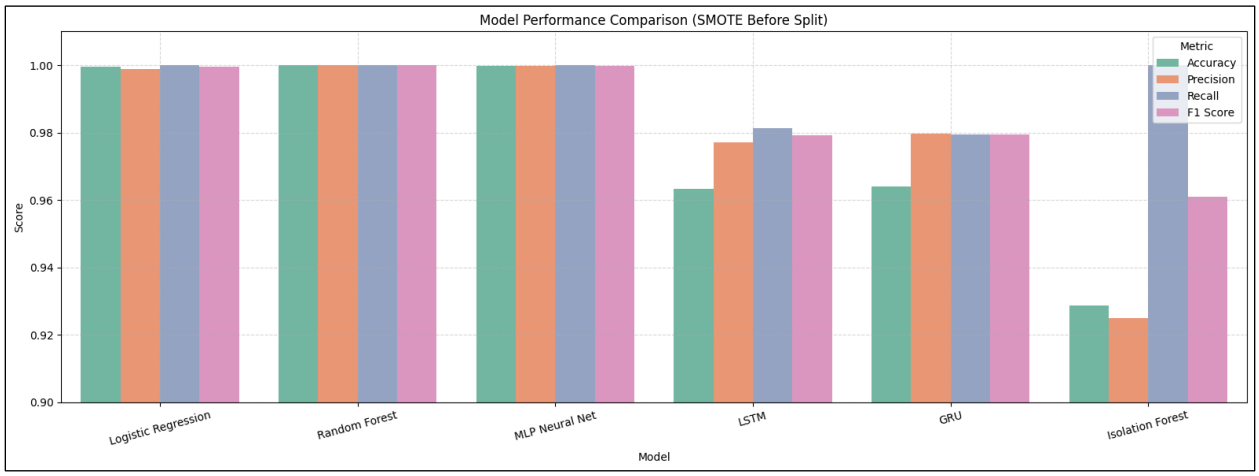
To support interpretability and comparison, we used the following visualizations:

- **Performance Table**: Tabular view of Accuracy, Precision, Recall, and F1 Score across all models and techniques.

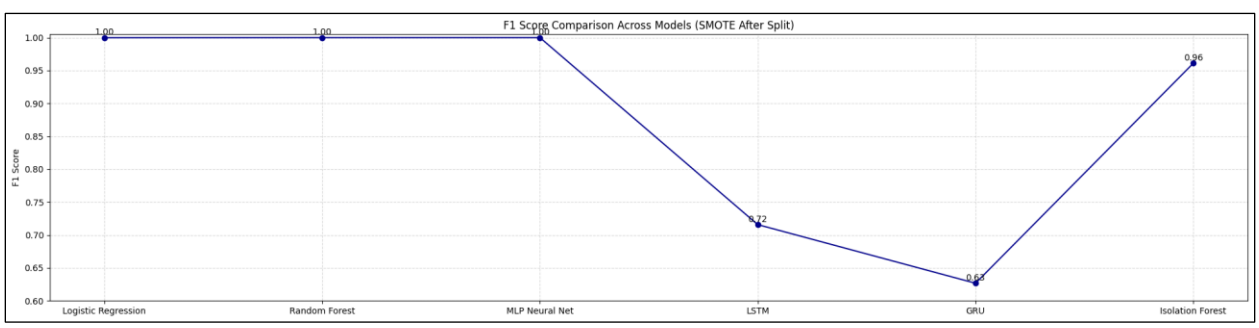
```
Model Performance Comparison (Manual Downsampling)

                     Accuracy  Precision  Recall  F1 Score
Logistic Regression    0.9995     0.9990  1.0000    0.9995
Random Forest          1.0000     1.0000  1.0000    1.0000
MLP Neural Net         1.0000     1.0000  1.0000    1.0000
LSTM                   0.4988     0.4968  0.6815    0.5747
GRU                    0.4984     0.4987  0.8027    0.6152
Isolation Forest       0.7367     0.6551  1.0000    0.7916
```

- **Bar Plot:** Plotted each model's metric scores for visual side-by-side comparison. This helped identify models with consistent high performance.



- **F1 Score Slope Graph:** Displayed F1 Scores across models to highlight which models had the most robust detection capability. This was effective in showing subtle differences, especially when values were close.



5. Summary and Finding

After implementing and evaluating multiple machine learning and deep learning models across four balancing techniques, the following key observations were made:

Supervised Models Showed High Consistency

- **Logistic Regression**, **Random Forest**, and **MLP Neural Net** consistently achieved near-perfect scores (≥ 0.99) across all techniques.
- These models proved highly effective for binary classification of HELLO_FLOOD attacks, demonstrating strong generalization even on unbalanced data (baseline).
- **Random Forest** in particular maintained 100% precision and recall in most scenarios, making it the most stable and performant model overall.

LSTM and GRU Performance Depended on Balancing

- While **LSTM** and **GRU** showed promising results (F1 Score ≈ 0.96–0.98) under balanced data conditions (SMOTE Before/After), their performance **dropped under Manual Downsampling**.
- This decline likely stems from the smaller training set size after downsampling, which is insufficient for deep learning models to learn temporal patterns effectively.

Isolation Forest Was Surprisingly Strong

- Despite being an **unsupervised model**, **Isolation Forest** achieved an **F1 Score of 0.9610** in all cases **except Manual Downsampling**.
- Its performance dropped in the **Downsampling technique (F1: 0.7916)** due to:
    - Reduced volume of benign data for model training.
    - Limited variability, making it harder for the model to isolate anomalies effectively.

Balancing Techniques Had Impact on Deep Models

- **SMOTE Before Split** produced the most **consistent performance** across all models, likely due to the availability of balanced data for both training and evaluation.
- **Manual Downsampling**, although useful for fair benchmarking, was less effective for LSTM/GRU due to reduced data size.
- **SMOTE After Split** preserved test set realism while improving training performance — an ideal compromise.

6. Conclusion

This project successfully addressed the problem of detecting **HELLO_FLOOD attacks** in RPL-based IoT networks using machine learning (ML), deep learning (DL), and unsupervised anomaly detection techniques.

By thoroughly analyzing the DETONAR research paper and working with the provided HELLO_FLOOD dataset, we built a robust classification pipeline that:

Cleaned and preprocessed real-world IoT network data.

- Applied **multiple class balancing techniques** (No SMOTE, SMOTE After Split, SMOTE Before Split, Manual Downsampling).
- Implemented a range of **supervised models**: Logistic Regression, Random Forest, MLP, LSTM, and GRU.
- Experimented with **Isolation Forest** for unsupervised anomaly detection.
- Conducted **comparative analysis** using performance metrics (Accuracy, Precision, Recall, F1 Score) and visual tools (confusion matrices, slope graphs).

The results showed:

- **Random Forest** was the most stable and accurate model across all techniques.
- **SMOTE Before Split** was the best balancing strategy for consistent performance.
- **Deep models (LSTM & GRU)** require sufficient training data, which makes them less effective in downsampled settings.
- **Isolation Forest** can work effectively without labels but is sensitive to data volume and variety.